

30-Day Agentic AI Engineering Roadmap

A complete, self-contained, beginner-friendly daily execution guide for transitioning into Agentic AI Engineering from an IT background.

Version 2.0 — All study links verified May 2026 | Tailored for Senior IT Support Engineers

Main Project Automation Support Agent Classifier + Tools + Approval Gate + Report	Mini Project Log Analysis Agent Parser + LLM RCA + Confidence Scoring	DevOps Layer Docker + GitHub Actions CI + HuggingFace Spaces Deployment	Interview Kit 20 Q&A; answers + 3 System Design Diagrams + Resume Bullets
--	---	--	--

Daily commitment: 2–3 hours | Suitable for: Python beginners, IT professionals new to AI

How to Use This Guide

This PDF is your **daily execution companion**. Every day has a goal, concepts to understand, resources to read, step-by-step instructions, commands to run, expected outputs, and interview questions to prepare. You do not need any other document.

Daily routine (follow this order every day):

- **1. Review (10 min)** — Open yesterday's notes. Check what was completed. Set today's single main goal.
- **2. Read (45 min)** — Read only today's study material. Take short notes in your own words.
- **3. Build (60–90 min)** — Implement exactly what you studied. Build small, test fast, keep momentum.
- **4. Debug (20–40 min)** — Fix errors. If stuck for 15 min, simplify the task and move forward.
- **5. Notes (20 min)** — Write what you learned, what broke, how you fixed it, how to explain it.
- **6. Commit (5 min)** — `git add . && git commit -m 'Day X: brief description' && git push`

Rule: Never spend more time reading than building. If you get stuck, try for 15 minutes, then simplify the task and move on.

Time budget per day:

- Normal days (concepts + small build): 2.5 hours
- Build-heavy days (Days 10–15, 19–20): 3 hours
- DevOps days (Days 22–25): 3–3.5 hours
- Polish/interview days (Days 28–30): 2 hours

Prerequisites:

- Python: can write basic scripts with functions, loops, and imports
- APIs: have called REST APIs before (not necessarily AI APIs)
- Tools: macOS or Linux machine with internet access
- Cost: OpenAI API — ~\$5–10 total for 30 days using gpt-3.5-turbo

Table of Contents

Week 1 — Foundation & Python Basics (Days 1–7)

- Day 01 — Environment Setup & Python Foundations
- Day 02 — LLM & Agent Concepts Deep Dive
- Day 03 — Prompt Engineering for IT Support
- Day 04 — First LLM API Calls in Python
- Day 05 — The Agent Loop — Understand & Map
- Day 06 — Interview Framing + Build Your First Chatbot
- Day 07 — Week 1 Review & First GitHub Push

Week 2 — Build the Automation Support Agent (Days 8–15)

- Day 08 — Choose Framework & Create Project Skeleton
- Day 09 — Define Enterprise Requirements
- Day 10 — Build the Issue Classifier
- Day 11 — Build Diagnostic Tools
- Day 12 — Connect Classifier to Tools — First End-to-End Run
- Day 13 — Build Structured Support Summaries
- Day 14 — Create Realistic Sample Cases
- Day 15 — Add Approval Gate (Human-in-the-Loop)

Week 3 — Polish Agent 1 + Build Log Analysis Agent (Days 16–21)

- Day 16 — Strengthen Prompts for Enterprise Use
- Day 17 — Add Evaluation Cases — Test Like an Engineer
- Day 18 — Start Project 2 — Log Analysis Agent
- Day 19 — Build the Log Parser
- Day 20 — Add LLM-Based RCA Summary
- Day 21 — Polish Both READMEs to Portfolio Standard

Week 4 — Docker, CI/CD, Deploy & Interview Prep (Days 22–30)

- Day 22 — Docker Basics — Containerise Your Thinking
- Day 23 — Run Project 1 in Docker
- Day 24 — Dockerize Project 2 — Log Analysis Agent
- Day 25 — GitHub Actions CI Pipeline
- Day 26 — Hugging Face Spaces — Choose Deployment Target
- Day 27 — Deploy One Project — Go Live
- Day 28 — Context Engineering Pass
- Day 29 — Interview & System Design Preparation
- Day 30 — Final Polish & Portfolio Pack

Week 1: Foundation & Python Basics

Days 1–7

D01 Environment Setup & Python Foundations

GOAL: Install every tool needed for the 30 days. Refresh core Python skills. Understand what Agentic AI is and why your IT background is a perfect match.

Core Concepts to Understand

- What is an LLM? (Large Language Model — a smart text-completion engine trained on billions of documents)
- What is an Agent? (an LLM that can observe a situation, decide on an action, call a tool, and iterate)
- What is RAG? (Retrieval-Augmented Generation — giving an LLM access to a knowledge base at query time)
- What is a Workflow? (a fixed sequence of steps, unlike an agent which decides its own steps)
- Why does your IT automation background matter? (You already think in: detect problem → run diagnostic → decide action → log result — that IS the agent loop)

Study Resources (read these first)

[Python Official Tutorial \(Beginner\)](https://docs.python.org/3/tutorial/index.html) — <https://docs.python.org/3/tutorial/index.html>

[HuggingFace AI Agents Course — Unit 0 Onboarding](https://huggingface.co/learn/agents-course/en/unit0/introduction) — <https://huggingface.co/learn/agents-course/en/unit0/introduction>

[What is an LLM? — IBM explainer](https://www.ibm.com/topics/large-language-models) — <https://www.ibm.com/topics/large-language-models>

Step-by-Step Instructions

1. Install Python 3.11+ from python.org — verify with: `python3 --version`
2. Install VS Code from code.visualstudio.com. Install the Python extension inside VS Code.
3. Install Git from git-scm.com — verify with: `git --version`
4. Install Docker Desktop from docker.com — verify with: `docker --version`
5. Create your main workspace and virtual environment (see commands below)
6. Create the folder structure shown below
7. Write notes in your own words for each concept listed above
8. Prepare your 2-minute self-introduction framing your IT background as AI-ready

Terminal Commands (run in order)

```
mkdir enterprise-agentic-ai
cd enterprise-agentic-ai
python3 -m venv .venv
source .venv/bin/activate # Mac/Linux
# .venv\Scripts\activate # Windows
pip install --upgrade pip
mkdir -p notes/projects/playground
touch notes/learning-journal.md
touch notes/interview-notes.md
touch notes/day01-foundations.md
code notes/day01-foundations.md
```

Expected Outputs by End of Day

OUTPUT: Working virtual environment (you see `.venv` in your terminal prompt)

OUTPUT: Folder structure: enterprise-agentic-ai / notes / projects /

OUTPUT: notes/day01-foundations.md with your own definitions of LLM, Agent, RAG, Workflow

OUTPUT: 2-minute self-intro written in notes/interview-notes.md

Interview Questions to Prepare

- ? What is the difference between an LLM and an AI Agent?
 - ? How does your IT automation background prepare you for Agentic AI Engineering?
 - ? What is RAG and when would you use it?
-

D02 LLM & Agent Concepts Deep Dive

GOAL: Understand how LLMs work, what tokens are, and map every agent concept to something you already do in IT support.

Core Concepts to Understand

- Tokens: LLMs don't read words, they read chunks called tokens (~0.75 words each). Cost is per token.
- Context window: the maximum tokens an LLM can 'see' at once (4K to 1M tokens depending on model)
- System prompt vs User prompt vs Assistant message — the three message roles
- Temperature: 0 = deterministic/safe, 1 = creative/risky. For IT support agents: always use 0 or 0.1
- The Agent Loop: Observe (read input) → Think (LLM reasons) → Act (call tool) → Observe result → repeat
- IT parallel: Ticket comes in → Diagnose → Run script → Read output → Decide next action = same loop

Study Resources (read these first)

[HuggingFace Agents Course — Unit 1: What is an Agent? —](https://huggingface.co/learn/agents-course/en/unit1/introduction)

<https://huggingface.co/learn/agents-course/en/unit1/introduction>

[HuggingFace Agents Course — Dummy Agent Library —](https://huggingface.co/learn/agents-course/en/unit1/dummy-agent-library)

<https://huggingface.co/learn/agents-course/en/unit1/dummy-agent-library>

[OpenAI: How LLMs work \(gentle intro\) — https://platform.openai.com/docs/introduction](https://platform.openai.com/docs/introduction)

Step-by-Step Instructions

1. Read HuggingFace Agents Course Unit 1 introduction (30 min)
2. In your notes, draw the agent loop as a flowchart (boxes and arrows, even hand-drawn is fine)
3. Map each agent loop step to a real IT support scenario you have lived through
4. Write the three message roles with an example of each from an IT support context
5. Read about temperature and write one sentence on why low temperature is right for IT agents

Terminal Commands (run in order)

```
touch notes/day02-agent-concepts.md
```

```
code notes/day02-agent-concepts.md
```

Expected Outputs by End of Day

OUTPUT: notes/day02-agent-concepts.md with: agent loop diagram, IT mapping, message role examples, temperature note

Interview Questions to Prepare

- ? Walk me through the agent observe-think-act loop with a concrete IT example.
 - ? Why is low temperature important for enterprise support agents?
 - ? What is a context window and why does it matter for log analysis?
-

D03 Prompt Engineering for IT Support

GOAL: Learn to write prompts that are precise, safe, and production-ready using your real IT scenarios.

Core Concepts to Understand

- A weak prompt gives the LLM too much freedom — it hallucinates or goes off-topic
- A strong prompt: sets role, provides context, specifies format, adds constraints
- Enterprise-safe prompt: tells the model what it CANNOT do (no guessing, no commands without approval)
- Prompt anatomy: [Role] + [Context] + [Task] + [Constraints] + [Output format]
- Few-shot prompting: give 2-3 examples inside the prompt to guide the model's output format

Study Resources (read these first)

[OpenAI Prompt Engineering Guide](https://platform.openai.com/docs/guides/prompt-engineering) — <https://platform.openai.com/docs/guides/prompt-engineering>

[OpenAI Help: Best Practices for Prompt Engineering](https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api) —

<https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>

Step-by-Step Instructions

1. Read OpenAI Prompt Engineering Guide (focus on: write clear instructions, provide reference text, split complex tasks)
2. Create 10 prompt pairs — one weak, one improved — using your real IT use cases listed below
3. For each improved prompt, label which element made it better: role / context / constraint / format
4. Write a reusable 'enterprise prompt template' you will use across all your projects

Reference Notes

Prompt pair topics to cover (one weak + one strong for each):

1. Low disk space alert on macOS endpoint
2. macOS upgrade failure at 75%
3. Citrix workspace not launching
4. Package install failure (pkg notarization error)
5. Application crash on launch
6. Network connectivity issue (intermittent)
7. User locked out of SSO
8. High CPU usage – unknown process
9. Printer not found after macOS update
10. VPN not connecting from a specific location

Terminal Commands (run in order)

```
touch notes/day03-prompts.md
code notes/day03-prompts.md
```

Expected Outputs by End of Day

OUTPUT: notes/day03-prompts.md with 10 prompt pairs and your enterprise prompt template

Interview Questions to Prepare

- ? What makes a prompt production-safe vs demo-only?
- ? Show me an example of a weak prompt and explain how you improved it.
- ? What is few-shot prompting and when would you use it in an IT support agent?

D04 First LLM API Calls in Python

GOAL: Make your first real API calls to an LLM. Understand the request/response structure. Build confidence before touching agent frameworks.

Core Concepts to Understand

- API key: a secret string that authenticates your code to the LLM provider — NEVER commit this to Git
- The messages array: list of dicts with 'role' and 'content' — this is how you build the conversation
- `response.choices[0].message.content` — where the model's reply lives in the JSON response
- `usage.total_tokens` — always log this to understand cost
- `.env` file: the standard way to store secrets locally without hardcoding them

Study Resources (read these first)

[OpenAI Python SDK — Getting Started](https://platform.openai.com/docs/libraries) — <https://platform.openai.com/docs/libraries>

[OpenAI API Reference — Chat Completions](https://platform.openai.com/docs/api-reference/chat) — <https://platform.openai.com/docs/api-reference/chat>

[python-dotenv docs](https://pypi.org/project/python-dotenv/) — <https://pypi.org/project/python-dotenv/>

Step-by-Step Instructions

1. Get an OpenAI API key from platform.openai.com (free tier works — use gpt-3.5-turbo to save cost)
2. Install required packages (see commands)
3. Create a `.env` file and store your key there — never hardcode it
4. Add `.env` to `.gitignore` immediately
5. Write `simple_completion.py` — a basic 'hello world' API call
6. Write `classify_issue.py` — sends an IT issue description, asks the model to classify it
7. Run both scripts and save sample outputs in your notes
8. Read and understand every field in the API response JSON

Terminal Commands (run in order)

```
pip install openai python-dotenv
touch .env .gitignore
echo 'OPENAI_API_KEY=your_key_here' >> .env
echo '.env' >> .gitignore
mkdir -p projects/playground
touch projects/playground/simple_completion.py
touch projects/playground/classify_issue.py
touch projects/playground/summarize_log.py
python projects/playground/simple_completion.py
python projects/playground/classify_issue.py
```

Code Samples

`simple_completion.py`

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
response = client.chat.completions.create(
    model="gpt-3.5-turbo",
```



```

messages=[
{"role": "system", "content": "You are a helpful IT support assistant."},
{"role": "user", "content": "My MacBook disk is showing 95% full. What should I check?"}
],
temperature=0.1,
)
print(response.choices[0].message.content)
print(f"Tokens used: {response.usage.total_tokens}")

```

classify_issue.py

```

import os
from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def classify_issue(description: str) -> str:
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": (
                "You are an IT support classifier. "
                "Classify the issue into exactly one of: "
                "DISK, UPGRADE, PACKAGING, APPLICATION, NETWORK, UNKNOWN. "
                "Reply with only the category word."
            )},
            {"role": "user", "content": description}
        ],
        temperature=0,
    )
    return response.choices[0].message.content.strip()

test_cases = [
    "My Mac says the startup disk is almost full",
    "Citrix Workspace crashes immediately on launch",
    "The macOS 14 upgrade is stuck at 75%",
]

for tc in test_cases:
    print(f"Issue: {tc}")
    print(f"Category: {classify_issue(tc)}\n")

```

Expected Outputs by End of Day

OUTPUT: Three working Python scripts that call the OpenAI API

OUTPUT: .env file with your API key (never committed to Git)

OUTPUT: Sample API responses saved in notes/day04-api-calls.md

OUTPUT: .gitignore protecting your secrets

Interview Questions to Prepare

- ? How do you secure API keys in a Python project?
- ? What is inside an OpenAI chat completions API response?
- ? Why should temperature be 0 for classification tasks?

D05 The Agent Loop — Understand & Map

GOAL: Deeply understand what makes an agent different from a script. Draw and explain the observe-think-act loop. Map it to enterprise IT support.

Core Concepts to Understand

- Agent vs Script: a script follows a fixed path; an agent decides its path based on observations
- Tool use: an agent can call external functions (tools) to get real data — this is what makes it powerful
- ReAct pattern (Reason + Act): the agent writes its reasoning, then writes its action, then observes
- Stopping conditions: agents need a way to decide 'I am done' — otherwise they loop forever
- Enterprise constraint: agents in production MUST have human approval for risky actions (the approval gate)

Study Resources (read these first)

[HuggingFace Agents Course — Unit 1: Tools](https://huggingface.co/learn/agents-course/en/unit1/tools) — <https://huggingface.co/learn/agents-course/en/unit1/tools>

[HuggingFace Agents Course — Agent Loop explained](https://huggingface.co/learn/agents-course/en/unit1/agent-steps-and-structure) —

<https://huggingface.co/learn/agents-course/en/unit1/agent-steps-and-structure>

[OpenAI Agents Guide](https://openai.github.io/openai-agents-python/) — <https://openai.github.io/openai-agents-python/>

Step-by-Step Instructions

1. Read HuggingFace Agents Course Unit 1: Tools section (30 min)
2. In your notes, write out a full ReAct-style trace for this scenario: 'User reports high CPU usage on their MacBook'
3. Draw the flow: Input → Classifier → Tool call → Tool result → Summariser → Output
4. List 5 tools your future Automation Support Agent will need
5. Write one paragraph: 'Why should an enterprise AI agent never run a cleanup script without approval?'

Terminal Commands (run in order)

```
touch notes/day05-agent-loop.md
code notes/day05-agent-loop.md
```

Expected Outputs by End of Day

OUTPUT: [notes/day05-agent-loop.md](#) with: ReAct trace, flow diagram (text), tool list, approval-gate paragraph

Interview Questions to Prepare

- ? What is the ReAct pattern and why is it useful for agent reasoning?
 - ? What is the difference between an agent and a workflow?
 - ? Why do enterprise AI agents need human approval gates?
-

GOAL: Turn your concepts into spoken interview answers. Build a working CLI chatbot — your first hands-on project.

Core Concepts to Understand

- A chatbot is NOT an agent: it has no tools, no loop, no decision-making — it just chats
- Conversation history: you must pass the full message history on every API call — the model has no memory
- System prompt defines the chatbot's personality and constraints for the whole session
- This chatbot is the foundation — Day 8 onwards you will extend it into a real agent

Study Resources (read these first)

[OpenAI Python SDK Quickstart](https://platform.openai.com/docs/libraries) — <https://platform.openai.com/docs/libraries>

[OpenAI Chat Completions API](https://platform.openai.com/docs/guides/text-generation) — <https://platform.openai.com/docs/guides/text-generation>

Step-by-Step Instructions

1. Write concise answers (60-90 seconds when spoken) for these 5 questions — practice saying them aloud
2. Create projects/chatbot/ and build chatbot.py (see code sample below)
3. Run the chatbot and test with at least 5 prompts from your Day 3 prompt file
4. Observe how the conversation history grows with each turn
5. Note: what breaks? what confuses the model? write it in your journal

Terminal Commands (run in order)

```
mkdir -p projects/chatbot
touch projects/chatbot/chatbot.py
python projects/chatbot/chatbot.py
```

Code Samples

chatbot.py

```
import os
from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
SYSTEM_PROMPT = (
    "You are an expert IT support assistant for a large enterprise. "
    "You help diagnose issues with macOS, networking, applications, and packages. "
    "Always ask for the OS version and affected application before diagnosing. "
    "Never suggest deleting system files. Keep responses under 150 words."
)
conversation = [{"role": "system", "content": SYSTEM_PROMPT}]
print("IT Support Chatbot (type 'quit' to exit)\n")
while True:
    user_input = input("You: ").strip()
    if user_input.lower() in ("quit", "exit"):
        break
    conversation.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(
```

```
model="gpt-3.5-turbo",
messages=conversation,
temperature=0.1,
)
reply = response.choices[0].message.content
conversation.append({"role": "assistant", "content": reply})
print(f"\nAssistant: {reply}\n")
```

Expected Outputs by End of Day

OUTPUT: [projects/chatbot/chatbot.py](#) — working CLI chatbot

OUTPUT: 5 interview answers written and practised aloud

OUTPUT: Session transcript saved in [notes/day06-chatbot-test.md](#)

Interview Questions to Prepare

- ? What is the difference between a chatbot and an agent?
 - ? How do you maintain conversation history when using the OpenAI API?
 - ? Why is a system prompt important in an enterprise AI application?
-

D07 Week 1 Review & First GitHub Push

GOAL: Consolidate everything from Week 1. Push all code to GitHub. Identify and fix any gaps before starting the main project build.

Core Concepts to Understand

- Git init, add, commit, push — the four commands you will use every day from now on
- README.md in the root: your first impression on any interviewer who visits your repo
- Week 1 checkpoint: can you define LLM, Agent, RAG, Workflow without looking at notes?

Study Resources (read these first)

[Git and GitHub for Beginners — freeCodeCamp](https://www.freecodecamp.org/news/git-and-github-for-beginners/) — <https://www.freecodecamp.org/news/git-and-github-for-beginners/>
[GitHub Quickstart](https://docs.github.com/en/get-started/quickstart/hello-world) — <https://docs.github.com/en/get-started/quickstart/hello-world>

Step-by-Step Instructions

1. Create a GitHub account if you do not have one (github.com)
2. Create a new repo called 'enterprise-agentic-ai' on GitHub (public)
3. Write a root README.md (see template below)
4. Push all Week 1 code and notes
5. Review all 5 days of notes — tighten any weak definitions
6. Record yourself answering these 5 questions without reading your notes (use your phone camera)
7. Watch the recording — identify what sounds unclear and fix it in writing

Reference Notes

README.md template for root:

```
# Enterprise Agentic AI Portfolio
> Built during a structured 30-day learning roadmap.
## Projects
- Automation Support Agent (Days 8-17)
- Log Analysis Agent (Days 18-20)
## Author
Senior IT Support Engineer transitioning into Agentic AI Engineering.
20 years of enterprise IT experience. Python, Docker, GitHub Actions.
```

Terminal Commands (run in order)

```
cd enterprise-agentic-ai
git init
touch README.md
git add .
git commit -m 'Week 1: foundations, chatbot, prompt examples'
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/enterprise-agentic-ai.git
git push -u origin main
```

Expected Outputs by End of Day

OUTPUT: Public GitHub repo with all Week 1 code and notes

OUTPUT: Root README.md explaining the repo

OUTPUT: notes/week1-review.md with honest self-assessment

OUTPUT: Video recording of your 5 interview answers

Interview Questions to Prepare

- ? Tell me about yourself and why you are moving into Agentic AI.
 - ? Define: LLM, Agent, RAG, Workflow — in plain English.
 - ? What did you build in your first week of learning?
-

Week 2: Build the Automation Support Agent

Days 8–15

D08 Choose Framework & Create Project Skeleton

GOAL: Compare agent frameworks. Choose OpenAI Agents SDK. Create the full project folder structure for the Automation Support Agent.

Core Concepts to Understand

- OpenAI Agents SDK: purpose-built for OpenAI models, clean Python API, built-in tracing, approval gates
- smolagents (HuggingFace): model-agnostic, very lightweight, good for learning; less enterprise-ready
- Why OpenAI Agents SDK for this project: better guardrails support, cleaner tool definition, production-closer
- Project structure matters: app.py (orchestration), tools.py (tool functions), prompts.py (all prompts), report.py (output formatting)

Study Resources (read these first)

[OpenAI Agents SDK — Overview](https://openai.github.io/openai-agents-python/) — <https://openai.github.io/openai-agents-python/>

[OpenAI Agents SDK — Quickstart](https://openai.github.io/openai-agents-python/quickstart/) — <https://openai.github.io/openai-agents-python/quickstart/>

[smolagents Documentation \(for comparison\)](https://huggingface.co/docs/smolagents/index) — <https://huggingface.co/docs/smolagents/index>

Step-by-Step Instructions

1. Read both framework overviews (30 min total — just the intro pages)
2. Write a 5-line comparison in your notes: OpenAI Agents SDK vs smolagents
3. Install the OpenAI Agents SDK
4. Create the full project skeleton with all files
5. Write the first line of each file — a docstring explaining its purpose
6. Commit the skeleton to GitHub

Terminal Commands (run in order)

```
pip install openai-agents
pip freeze > requirements.txt
mkdir -p projects/automation_support_agent/sample_data
touch projects/automation_support_agent/app.py
touch projects/automation_support_agent/tools.py
touch projects/automation_support_agent/prompts.py
touch projects/automation_support_agent/report.py
touch projects/automation_support_agent/requirements.md
touch projects/automation_support_agent/README.md
touch projects/automation_support_agent/eval_cases.md
git add . && git commit -m 'Day 8: project skeleton for Automation Support Agent'
```

Expected Outputs by End of Day

OUTPUT: projects/automation_support_agent/ with 7 files created

OUTPUT: requirements.txt updated

OUTPUT: notes/day08-framework-choice.md with your framework comparison

Interview Questions to Prepare

- ? Why did you choose the OpenAI Agents SDK over LangChain or smolagents?
 - ? What does each file in your project skeleton do?
 - ? What is the difference between a tool and a prompt in an agent system?
-

D09 Define Enterprise Requirements

GOAL: Write a proper MVP specification for the Automation Support Agent — like a real internal product document.

Core Concepts to Understand

- MVP (Minimum Viable Product): the smallest version that proves the concept and is safe to demo
- Safe tools vs risky tools: read-only diagnostics (safe) vs cleanup/delete/reinstall (risky, needs approval)
- Structured output: always return JSON with fixed fields — makes the output machine-readable and testable
- Issue categories define the routing logic: the classifier sends each ticket to the right diagnostic tool

Study Resources (read these first)

[OpenAI Agents Guide — Guardrails](https://openai.github.io/openai-agents-python/guardrails/) — <https://openai.github.io/openai-agents-python/guardrails/>

[OpenAI Agents Guide — Running Agents](https://openai.github.io/openai-agents-python/running_agents/) — https://openai.github.io/openai-agents-python/running_agents/

Step-by-Step Instructions

1. Open `projects/automation_support_agent/requirements.md`
2. Define 5 issue categories with examples (see template below)
3. List all tools — mark each as SAFE (auto-run) or RISKY (needs approval)
4. Define the output schema (JSON fields the agent must always produce)
5. Write acceptance criteria: how will you know the MVP is working?

Reference Notes

Issue Categories to define:

```
DISK — low disk space, storage full, large file warnings
UPGRADE — macOS upgrade stuck/failed, version mismatch
PACKAGING — pkg install failure, notarization error, code signing issue
APPLICATION — app crash, won't launch, license error (e.g. Citrix, Zoom)
UNKNOWN — cannot classify with confidence
```

Required JSON output fields:

```
issue_category, confidence_score, tools_invoked,
findings, recommendation, approval_required, summary
```

Terminal Commands (run in order)

```
code projects/automation_support_agent/requirements.md
```

Expected Outputs by End of Day

OUTPUT: `projects/automation_support_agent/requirements.md` — complete MVP spec

OUTPUT: Committed to GitHub

Interview Questions to Prepare

- ? What is the difference between an MVP and a production-ready agent?
 - ? How do you decide which agent actions need human approval?
 - ? Why is structured JSON output important for enterprise AI agents?
-

D10 Build the Issue Classifier

GOAL: Implement the first real component of your agent — the issue classifier that routes every support ticket.

Core Concepts to Understand

- Classification is the entry point: every agent action starts with understanding what type of problem it is
- Confidence score: always return a number (0.0–1.0) indicating how sure the model is
- Edge cases: ambiguous tickets, multiple issues in one description — handle gracefully with UNKNOWN
- The classifier output feeds the routing logic — it determines which tool gets called next

Study Resources (read these first)

[OpenAI Prompt Engineering — Classification tasks](https://platform.openai.com/docs/guides/prompt-engineering) — <https://platform.openai.com/docs/guides/prompt-engineering>

[OpenAI Structured Outputs](https://platform.openai.com/docs/guides/structured-outputs) — <https://platform.openai.com/docs/guides/structured-outputs>

Step-by-Step Instructions

1. Implement the `classify_issue()` function in `app.py`
2. The function takes a free-text ticket description and returns a dict with category + confidence
3. Test with 10 sample inputs — include edge cases (vague, multi-issue, non-IT)
4. Add logging so you can see what the model returns for each input
5. Write failing test cases in `eval_cases.md`

Terminal Commands (run in order)

```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 10: issue classifier working'
```

Code Samples

app.py (classifier)

```
import os, json
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
CATEGORIES = ["DISK", "UPGRADE", "PACKAGING", "APPLICATION", "NETWORK", "UNKNOWN"]

def classify_issue(description: str) -> dict:
    prompt = f"""You are an IT support classifier for a large enterprise.
    Classify the following support ticket into exactly one category.
    Categories: {' '.join(CATEGORIES)}
    Reply ONLY with valid JSON. No explanation.
    Format: {{"category": "DISK", "confidence": 0.95, "reason": "one line"}}
    Ticket: {description}"""
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
    )
    return json.loads(response.choices[0].message.content)
```

```
# Test
samples = [
    "My MacBook Pro says the startup disk is almost full",
    "Citrix Workspace crashes immediately when I try to launch it",
    "The macOS Sonoma upgrade is stuck at 75% and won't proceed",
    "I can't install the company VPN package, says notarization failed",
    "My laptop is slow", # ambiguous
]
for s in samples:
    result = classify_issue(s)
    print(f"Input: {s}")
    print(f"Result: {result}\n")
```

Expected Outputs by End of Day

OUTPUT: Working `classify_issue()` function in `app.py`

OUTPUT: 10 test cases run and results noted

OUTPUT: `eval_cases.md` started with first 10 cases

Interview Questions to Prepare

- ? How does your classifier handle ambiguous or multi-issue tickets?
 - ? Why do you return a confidence score alongside the category?
 - ? What happens if the model returns malformed JSON?
-

D11 Build Diagnostic Tools

GOAL: Create the tool layer — safe, deterministic functions the agent will call to gather evidence before recommending action.

Core Concepts to Understand

- Tools must be deterministic: same input → same output. No surprises in enterprise environments.
- Mock tools first: use realistic fake data so you can test the full agent loop without real system access
- Structured return: every tool returns a dict, not a string — the agent needs to parse and reason over it
- Tool documentation: the agent reads the function docstring to decide when to call it — write good docstrings

Study Resources (read these first)

[OpenAI Agents SDK — Tools](https://openai.github.io/openai-agents-python/tools/) — <https://openai.github.io/openai-agents-python/tools/>

[Python type hints and docstrings — Real Python](https://realpython.com/python-type-checking/) — <https://realpython.com/python-type-checking/>

Step-by-Step Instructions

1. Implement all 5 tool functions in tools.py (see code sample below)
2. Each function must have: type hints, a clear docstring, structured dict return
3. Test each function independently with `python -c 'from tools import ...'`
4. Add a `TOOL_REGISTRY` dict at the bottom mapping category names to tool functions

Terminal Commands (run in order)

```
code projects/automation_support_agent/tools.py
python -c "from projects.automation_support_agent.tools import check_disk_space;
print(check_disk_space())"
git add . && git commit -m 'Day 11: diagnostic tools layer'
```

Code Samples

tools.py

```
"""Diagnostic tools for the Automation Support Agent.
Each tool is safe (read-only) and returns structured data.
"""

import random, datetime

def check_disk_space() -> dict:
    """Check available disk space on the primary volume. Safe, read-only."""
    return {
        "volume": "Macintosh HD",
        "total_gb": 512,
        "used_gb": 487,
        "free_gb": 25,
        "percent_used": 95.1,
        "critical": True,
        "checked_at": datetime.datetime.now().isoformat(),
    }

def check_os_version() -> dict:
    """Return the current macOS version and pending updates. Safe, read-only."""
    return {
        "os_name": "macOS",
```

```

"version": "14.2.1",
"build": "23C71",
"pending_update": "14.4",
"update_required": True,
"last_checked": datetime.datetime.now().isoformat(),
}

def check_package_status(package_name: str = "CompanyVPN") -> dict:
    """Check installation status of a named package. Safe, read-only."""
    return {
        "package": package_name,
        "installed": False,
        "last_attempt": "2024-01-15T09:23:11",
        "error_code": "ERR_NOTARIZATION_FAILED",
        "error_detail": "Package not notarized by Apple. Contact IT admin.",
        "requires_approval": True,
    }

def check_running_processes() -> dict:
    """List top CPU-consuming processes. Safe, read-only."""
    return {
        "top_processes": [
            {"pid": 1234, "name": "kernel_task", "cpu_pct": 45.2},
            {"pid": 5678, "name": "Citrix Workspace", "cpu_pct": 38.1},
            {"pid": 9012, "name": "mds_stores", "cpu_pct": 12.4},
        ],
        "total_cpu_usage_pct": 95.7,
        "sampled_at": datetime.datetime.now().isoformat(),
    }

def read_sample_log(path: str = "sample.log") -> dict:
    """Read a log file and return last 20 lines with error count. Safe, read-only."""
    sample_lines = [
        "2024-01-15 09:22:00 INFO Starting upgrade process...",
        "2024-01-15 09:22:05 INFO Downloading macOS 14.4...",
        "2024-01-15 09:35:22 ERROR Download failed: disk full",
        "2024-01-15 09:35:23 ERROR Upgrade aborted: insufficient space",
        "2024-01-15 09:35:24 WARN Rolling back to previous state",
    ]
    errors = [l for l in sample_lines if "ERROR" in l]
    return {
        "path": path,
        "total_lines": len(sample_lines),
        "error_count": len(errors),
        "errors": errors,
        "last_lines": sample_lines[-5:],
    }

TOOL_REGISTRY = {
    "DISK": check_disk_space,
    "UPGRADE": check_os_version,
    "PACKAGING": check_package_status,
    "APPLICATION": check_running_processes,
    "NETWORK": check_running_processes,
}

```

Expected Outputs by End of Day

OUTPUT: projects/automation_support_agent/tools.py with 5 tool functions + TOOL_REGISTRY
OUTPUT: All tools tested individually

Interview Questions to Prepare

- ? Why should agent tools be read-only first before adding remediation actions?
 - ? How does the agent know which tool to call for a given issue type?
 - ? Why return a dict instead of a plain string from a tool function?
-

D12 Connect Classifier to Tools — First End-to-End Run

GOAL: Wire the classifier output to the tool layer. Run your first complete support ticket through the system end-to-end.

Core Concepts to Understand

- Routing logic: use the classifier's category output as a key to look up the right tool in TOOL_REGISTRY
- Error handling: the classifier might return UNKNOWN or invalid JSON — handle both gracefully
- End-to-end trace: for every ticket you process, print each step so you can see the agent 'thinking'
- This is the most important day of Week 2 — getting end-to-end flow is the foundation for everything after

Study Resources (read these first)

[OpenAI Agents SDK — Running Agents](https://openai.github.io/openai-agents-python/running_agents/) — https://openai.github.io/openai-agents-python/running_agents/

Step-by-Step Instructions

1. Add a process_ticket() function in app.py that: classifies → routes → calls tool → returns combined result
2. Add print statements at each step so you can see the flow
3. Test with 5 different ticket types — one for each category
4. Handle the UNKNOWN category gracefully (no tool call, return safe message)
5. Commit working end-to-end flow

Terminal Commands (run in order)

```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 12: end-to-end classifier-to-tool flow'
```

Code Samples

app.py (process_ticket)

```
from tools import TOOL_REGISTRY

def process_ticket(description: str) -> dict:
    print(f"\n--- Processing Ticket ---")
    print(f"Input: {description}")

    # Step 1: Classify
    classification = classify_issue(description)
    category = classification.get("category", "UNKNOWN")
    confidence = classification.get("confidence", 0)
    print(f"Category: {category} (confidence: {confidence})")

    # Step 2: Route to tool
    if category == "UNKNOWN" or confidence < 0.6:
        print("Routing to: manual review (low confidence)")
        tool_result = {"status": "needs_human_review",
                      "reason": "Could not classify with sufficient confidence"}
    else:
        tool_fn = TOOL_REGISTRY.get(category)
        print(f"Routing to: {tool_fn.__name__}")
        tool_result = tool_fn()
        print(f"Tool result: {tool_result}")

    return {
        "ticket": description,
```

```
"classification": classification,
"tool_result": tool_result,
}
if __name__ == "__main__":
tickets = [
    "My MacBook Pro says the startup disk is almost full",
    "Citrix Workspace crashes immediately on launch",
    "macOS Sonoma upgrade is stuck at 75%",
    "VPN package installation fails with notarization error",
    "My computer feels slow today",
]
for ticket in tickets:
    result = process_ticket(ticket)
    print(f"Final result keys: {list(result.keys())}\n")
```

Expected Outputs by End of Day

OUTPUT: End-to-end flow working for all 5 ticket types

OUTPUT: Console output showing each step of the process

Interview Questions to Prepare

- ? Walk me through what happens when a ticket enters your Automation Support Agent.
 - ? How do you handle low-confidence classifications?
 - ? What would you change if you needed to support 20 issue categories instead of 5?
-

D13 Build Structured Support Summaries

GOAL: Add the report layer — transform raw tool output into a clean, structured support summary that a human engineer can act on.

Core Concepts to Understand

- Structured output is the difference between a demo and a production tool: machines can parse it, humans can read it
- The LLM adds value here by interpreting raw tool data into plain-English findings and recommendations
- Always separate data gathering (tools) from interpretation (LLM) — this makes each part testable
- JSON output with fixed fields means your agent can be integrated into ticketing systems (Jira, ServiceNow)

Study Resources (read these first)

[OpenAI Structured Outputs guide](https://platform.openai.com/docs/guides/structured-outputs) — <https://platform.openai.com/docs/guides/structured-outputs>

Step-by-Step Instructions

1. Implement `generate_report()` in `report.py`
2. The function takes: ticket text + classification + `tool_result`, calls the LLM to interpret, returns structured JSON
3. Save 3 sample JSON outputs in `projects/automation_support_agent/sample_data/`
4. Update `process_ticket()` in `app.py` to call `generate_report()` as the final step

Terminal Commands (run in order)

```
code projects/automation_support_agent/report.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 13: structured report generation'
```

Code Samples

report.py

```
import json, os
from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
def generate_report(ticket: str, classification: dict,
tool_result: dict) -> dict:
    """Generate a structured support summary from evidence."""
    prompt = f"""You are an enterprise IT support engineer writing a support summary.
    Use ONLY the evidence provided. Do not guess or assume.
    If information is missing, say 'Not available'.
    Reply ONLY with valid JSON matching this exact schema:
    {{
    "issue_category": "...",
    "severity": "LOW|MEDIUM|HIGH|CRITICAL",
    "findings": "2-3 sentences describing what was found",
    "root_cause": "one sentence root cause or 'Inconclusive'",
    "recommendation": "specific next step for the engineer",
    "approval_required": true or false,
    "approval_reason": "why approval is needed or null",
    "summary": "one sentence executive summary"
    }}
```

```
}}
Ticket: {ticket}
Classification: {json.dumps(classification)}
Tool Evidence: {json.dumps(tool_result)}"""
response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": prompt}],
    temperature=0,
)
return json.loads(response.choices[0].message.content)
```

Expected Outputs by End of Day

OUTPUT: report.py with generate_report() function

OUTPUT: 3 sample JSON output files in sample_data/

OUTPUT: Full pipeline: ticket → classify → tool → report

Interview Questions to Prepare

- ? Why do you separate tool execution from LLM interpretation in your agent?
 - ? How do you ensure the LLM only uses evidence it was given, not hallucinated data?
 - ? How could your structured output be integrated with ServiceNow or Jira?
-

D14 Create Realistic Sample Cases

GOAL: Build a library of realistic sample inputs based on your 20 years of IT experience. These make the project credible in interviews.

Core Concepts to Understand

- Sanitised real data: use patterns from real incidents but remove all personal/company-specific details
- Diverse samples: cover all 5 categories, varying severity levels, and edge cases
- Good sample data is what separates a portfolio project from a toy demo
- Reviewers and interviewers will test your agent with unexpected inputs — prepare for that

Study Resources (read these first)

[Writing good test cases — Google Testing Blog](https://testing.googleblog.com/) — <https://testing.googleblog.com/>

Step-by-Step Instructions

1. Create 4 realistic sample log/input files in `sample_data/`
2. Run the full pipeline against each sample file
3. Note any failures or weak outputs — these become Day 17 eval cases
4. Create 5 more edge-case tickets (vague, multi-issue, non-English, etc.)

Reference Notes

Sample `disk_issue.txt` content:

```
User reports: MacBook Pro 16-inch (2021, M1 Max) running macOS 14.2.1
Error: 'Your disk is almost full' notification appeared during Zoom call.
Cannot save files. Applications running slowly.
Last cleanup was 6 months ago. No recent large downloads known.
```

Sample `upgrade_failure.log` content:

```
2024-01-15 08:00 INFO Starting macOS 14.4 upgrade
2024-01-15 08:45 INFO Download complete (12.4 GB)
2024-01-15 09:12 ERROR Preparation failed: not enough space on Macintosh HD
2024-01-15 09:12 ERROR Required: 35 GB, Available: 22 GB
2024-01-15 09:13 WARN Upgrade cancelled. System restored.
```

Terminal Commands (run in order)

```
code projects/automation_support_agent/sample_data/disk_issue.txt
code projects/automation_support_agent/sample_data/upgrade_failure.log
code projects/automation_support_agent/sample_data/package_install.log
code projects/automation_support_agent/sample_data/citrix_issue.log
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 14: realistic sample cases added'
```

Expected Outputs by End of Day

OUTPUT: 4 sample data files in `sample_data/`

OUTPUT: Pipeline test results for all 4 samples

OUTPUT: Failure notes in `notes/day14-sample-test.md`

Interview Questions to Prepare

- ? Why does realistic sample data matter for a portfolio project?
- ? How did you sanitise real IT incident data for this project?

? What edge cases did your agent struggle with and how did you address them?

D15 Add Approval Gate (Human-in-the-Loop)

GOAL: Implement the safety model that is non-negotiable in enterprise automation — human approval before any risky action.

Core Concepts to Understand

- HITL (Human-in-the-Loop): the pattern where an agent pauses and asks a human before proceeding
- Safe actions (auto-run): reading logs, checking disk, listing processes — no risk
- Risky actions (approval required): deleting files, reinstalling packages, running remediation scripts
- This is your strongest interview talking point: enterprise AI without guardrails is unusable
- The approval gate is the difference between a demo agent and a production agent

Study Resources (read these first)

[OpenAI Agents SDK — Guardrails & Human Review](https://openai.github.io/openai-agents-python/guardrails/) — <https://openai.github.io/openai-agents-python/guardrails/>
[OpenAI: Safety in building agents](https://developers.openai.com/api/docs/guides/agents) — <https://developers.openai.com/api/docs/guides/agents>

Step-by-Step Instructions

1. Add `SAFE_TOOLS` and `RISKY_TOOLS` lists to `tools.py`
2. Add `approval_gate()` function to `app.py` — prompts the user for confirmation
3. Update `process_ticket()` to check tool risk level before execution
4. Test: risky action should pause and ask for approval; safe action should run automatically
5. The `approval_required` field in your report JSON should reflect this logic

Terminal Commands (run in order)

```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 15: approval gate added – human-in-the-loop'
```

Code Samples

app.py (approval gate)

```
SAFE_ACTIONS = ["check_disk_space", "check_os_version",
                "check_running_processes", "read_sample_log"]
RISKY_ACTIONS = ["reinstall_package", "delete_cache",
                "force_reboot", "run_cleanup_script"]

def approval_gate(action_name: str, details: str) -> bool:
    """Pause and ask human for approval before a risky action."""
    print(f"\n[APPROVAL REQUIRED]")
    print(f"Action : {action_name}")
    print(f"Details: {details}")
    answer = input("Approve? (yes/no): ").strip().lower()
    return answer in ("yes", "y")

def is_risky(tool_name: str) -> bool:
    return tool_name in RISKY_ACTIONS
```

Expected Outputs by End of Day

OUTPUT: Approval gate working — tested with both safe and risky actions

OUTPUT: approval_required field correctly set in all report outputs

Interview Questions to Prepare

- ? Why should an AI agent never automatically run remediation scripts without approval?
 - ? How did you implement human-in-the-loop in your Automation Support Agent?
 - ? How would you extend the approval gate to use Slack or email notifications?
-

D16 Strengthen Prompts for Enterprise Use

GOAL: Harden all prompts to reduce hallucination risk and make the agent safe for real enterprise use.

Core Concepts to Understand

- Hallucination: the LLM confidently states something false — catastrophic in IT automation
- Evidence-only constraint: the prompt must explicitly forbid using information not in the provided evidence
- Safe fallback: the agent must say 'I don't know' rather than guess — this must be written into the prompt
- Prefer safest action: when unsure, always recommend the lowest-risk next step
- Prompt versioning: keep a changelog of prompt changes and what they improved

Study Resources (read these first)

[OpenAI Prompt Engineering — Reduce hallucinations](https://platform.openai.com/docs/guides/prompt-engineering) — <https://platform.openai.com/docs/guides/prompt-engineering>
[HuggingFace Context Course — Unit 1](https://huggingface.co/learn/context-course/unit0/introduction) — <https://huggingface.co/learn/context-course/unit0/introduction>

Step-by-Step Instructions

1. Review all prompts in prompts.py
2. Add these constraints to every prompt: evidence-only, safe fallback, no guessing
3. Add a 'confidence threshold' prompt: if not confident, say so and recommend human review
4. Test the hardened prompts with ambiguous inputs — verify the fallback behaviour works
5. Document each prompt change with a before/after comparison

Terminal Commands (run in order)

```
code projects/automation_support_agent/prompts.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 16: enterprise-safe prompts with anti-hallucination constraints'
```

Expected Outputs by End of Day

OUTPUT: prompts.py with all prompts hardened and documented

OUTPUT: Test results showing safe fallback behaviour for ambiguous inputs

Interview Questions to Prepare

- ? How do you reduce hallucination risk in an IT support agent?
- ? What happens in your agent when evidence is insufficient to make a recommendation?
- ? What is context engineering and how does it relate to prompt design?

D17 Add Evaluation Cases — Test Like an Engineer

GOAL: Evaluate the Automation Support Agent systematically. 15 test cases with expected outcomes. This separates engineers from demo builders.

Core Concepts to Understand

- Evaluation is not optional: if you cannot measure quality, you cannot improve it
- Expected output: for each test case, define what the correct category, tool, and recommendation SHOULD be
- Pass/fail scoring: run each case, compare actual vs expected, record the result
- This eval sheet becomes your proof of quality in interviews

Study Resources (read these first)

[OpenAI Evals — Evaluate agent workflows](https://developers.openai.com/api/docs/guides/agents) — <https://developers.openai.com/api/docs/guides/agents>

[How to evaluate LLM outputs — Weights & Biases guide](https://wandb.ai/site/articles/evaluate-llm) — <https://wandb.ai/site/articles/evaluate-llm>

Step-by-Step Instructions

1. Open eval_cases.md and write 15 test cases
2. For each case: input ticket, expected_category, expected_tool, acceptable_recommendation
3. Run each case manually and record: actual_category, actual_recommendation, pass/fail
4. Calculate a pass rate. If below 80%, identify the top 2 failure patterns and fix them
5. Commit the completed eval sheet

Reference Notes

Eval table columns:

#	Ticket	Expected Category	Expected Tool	Actual Category	Pass?	Notes
---	--------	-------------------	---------------	-----------------	-------	-------

Include these tricky cases:

- 'My computer is hot' (ambiguous — could be DISK or APPLICATION or UNKNOWN)
- 'Error code 11001' (no context — should return UNKNOWN)
- 'Everything is broken' (vague — should escalate to human)

Terminal Commands (run in order)

```
code projects/automation_support_agent/eval_cases.md
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 17: 15 eval cases with pass/fail results'
```

Expected Outputs by End of Day

OUTPUT: eval_cases.md with 15 cases and pass/fail results

OUTPUT: Pass rate calculated. Failures documented with fix notes.

Interview Questions to Prepare

- ? How do you measure the quality of an AI agent?
- ? What was your pass rate on your eval suite and what did you do about failures?
- ? How is evaluating an LLM agent different from evaluating a traditional script?

D18 Start Project 2 — Log Analysis Agent

GOAL: Begin the second portfolio project: an agent that reads automation logs and produces root cause analysis. Directly tied to your IT background.

Core Concepts to Understand

- Log analysis is one of the highest-value AI applications in enterprise IT
- Two-stage design: deterministic pattern detection FIRST, then LLM interpretation SECOND
- Why two stages? Deterministic parsing is fast, cheap, and reliable. LLM adds the 'why' and 'what next'.
- Hybrid design (deterministic + LLM) is more robust than pure LLM — your key differentiator

Study Resources (read these first)

[HuggingFace Context Course — Context Engineering](https://huggingface.co/learn/context-course/unit0/introduction) — <https://huggingface.co/learn/context-course/unit0/introduction>
[Python re module — Regular Expressions](https://docs.python.org/3/library/re.html) — <https://docs.python.org/3/library/re.html>

Step-by-Step Instructions

1. Create the full project skeleton for log_analysis_agent
2. Write the problem statement in README.md — what problem does this agent solve?
3. Define the output schema: probable_root_cause, confidence, affected_components, next_action
4. Create 5 realistic sample log files (use patterns from your real experience)
5. Commit the skeleton

Terminal Commands (run in order)

```
mkdir -p projects/log_analysis_agent/sample_logs
touch projects/log_analysis_agent/app.py
touch projects/log_analysis_agent/parser.py
touch projects/log_analysis_agent/prompts.py
touch projects/log_analysis_agent/README.md
touch projects/log_analysis_agent/sample_logs/upgrade_fail.log
touch projects/log_analysis_agent/sample_logs/disk_full.log
touch projects/log_analysis_agent/sample_logs/signing_error.log
touch projects/log_analysis_agent/sample_logs/permission_denied.log
touch projects/log_analysis_agent/sample_logs/timeout.log
git add . && git commit -m 'Day 18: Log Analysis Agent skeleton'
```

Expected Outputs by End of Day

OUTPUT: projects/log_analysis_agent/ with all files created

OUTPUT: README.md with clear problem statement

OUTPUT: 5 sample log files with realistic content

Interview Questions to Prepare

- ? Why did you build a Log Analysis Agent as your second project?
- ? What is the advantage of combining deterministic parsing with LLM reasoning?
- ? How does this project relate to your existing IT support experience?

D19 Build the Log Parser

GOAL: Implement deterministic pattern detection in the log parser. Find failure patterns before the LLM sees anything.

Core Concepts to Understand

- Regex patterns: powerful for extracting structured information from unstructured log text
- Pattern labels: PERMISSION_DENIED, DISK_FULL, TIMEOUT, SIGNING_ERROR, PACKAGE_FAILURE, UPGRADE_ABORT
- Return line numbers: interviewers will ask 'how do you know where the error is?' — you return the exact line
- Deterministic first: if the pattern matches, you know with certainty. LLM handles the interpretation.

Study Resources (read these first)

[Python re module documentation](https://docs.python.org/3/library/re.html) — <https://docs.python.org/3/library/re.html>

[Regex tutorial for Python — Real Python](https://realpython.com/regex-python/) — <https://realpython.com/regex-python/>

Step-by-Step Instructions

1. Implement `parse_log()` in `parser.py` using regex patterns
2. Support at least 6 failure patterns
3. Return: `matched_patterns` (list of dicts with `pattern`, `line_number`, `matched_text`)
4. Test on all 5 sample logs
5. Count matches, no matches, partial matches — note each

Terminal Commands (run in order)

```
code projects/log_analysis_agent/parser.py
python projects/log_analysis_agent/parser.py
git add . && git commit -m 'Day 19: log parser with pattern detection'
```

Code Samples

parser.py

```
"""Deterministic log parser for the Log Analysis Agent.
Detects known failure patterns before LLM analysis.
"""

import re
from typing import List, Dict

PATTERNS = {
    "PERMISSION_DENIED": r"permission denied|Operation not permitted",
    "DISK_FULL": r"disk full|no space left|insufficient space|not enough space",
    "TIMEOUT": r"timed out|timeout|connection refused|deadline exceeded",
    "SIGNING_ERROR": r"notarization|code sign|codesign|ERR_SIGNING",
    "PACKAGE_FAILURE": r"package failed|install failed|ERR_INSTALL|pkg error",
    "UPGRADE_ABORT": r"upgrade (cancelled|aborted|failed)|upgrade error",
    "AUTH_FAILURE": r"authentication failed|invalid credentials|ERR_AUTH",
}

def parse_log(log_text: str) -> dict:
    """Parse log text and return detected failure patterns with line numbers."""
    lines = log_text.strip().split("\n")
    matches = []
    for pattern_name, pattern_regex in PATTERNS.items():
```

```

for i, line in enumerate(lines, start=1):
    if re.search(pattern_regex, line, re.IGNORECASE):
        matches.append({
            "pattern": pattern_name,
            "line_number": i,
            "matched_text": line.strip(),
        })
return {
    "total_lines": len(lines),
    "matched_patterns": matches,
    "pattern_names": list({m["pattern"] for m in matches}),
    "error_count": len(matches),
    "has_errors": len(matches) > 0,
}
if __name__ == "__main__":
    sample = """
2024-01-15 09:12 ERROR Preparation failed: not enough space on Macintosh HD
2024-01-15 09:12 ERROR Upgrade aborted: insufficient space
2024-01-15 09:13 WARN Rolling back changes
"""
    result = parse_log(sample)
    import json
    print(json.dumps(result, indent=2))

```

Expected Outputs by End of Day

OUTPUT: parser.py with parse_log() function and 7 patterns

OUTPUT: All 5 sample logs tested with results

Interview Questions to Prepare

- ? Why do you use regex pattern matching before sending log data to an LLM?
- ? How do you handle log formats that don't match your expected patterns?
- ? What is the performance advantage of deterministic parsing over pure LLM?

D20 Add LLM-Based RCA Summary

GOAL: Use parser evidence + LLM reasoning to generate a concise Root Cause Analysis. Complete the Log Analysis Agent.

Core Concepts to Understand

- RCA (Root Cause Analysis): not just 'what failed' but 'why it failed' and 'how to prevent recurrence'
- Evidence-grounded LLM: pass the parser results into the prompt so the LLM reasons over real evidence
- Confidence from parser: if many patterns match, confidence is high; zero matches → low confidence
- This is the hybrid design in action: parser provides certainty, LLM provides intelligence

Study Resources (read these first)

[OpenAI API — Chat Completions](https://platform.openai.com/docs/guides/text-generation) — <https://platform.openai.com/docs/guides/text-generation>

[HuggingFace Context Course — Structuring context for agents](https://huggingface.co/learn/context-course/unit0/introduction) — <https://huggingface.co/learn/context-course/unit0/introduction>

Step-by-Step Instructions

1. Implement `generate_rca()` in `app.py` — takes log text, runs parser, then calls LLM
2. The LLM prompt includes: parser results, raw log excerpt, RCA schema
3. Return: `probable_root_cause`, confidence (HIGH/MEDIUM/LOW), `affected_component`, `next_action`, prevention
4. Save 3 sample RCA outputs in `sample_logs/`
5. Test with a log that has zero pattern matches — verify graceful handling

Terminal Commands (run in order)

```
code projects/log_analysis_agent/app.py
python projects/log_analysis_agent/app.py
git add . && git commit -m 'Day 20: LLM RCA complete – Log Analysis Agent working'
```

Code Samples

app.py (RCA)

```
import os, json
from openai import OpenAI
from dotenv import load_dotenv
from parser import parse_log

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def generate_rca(log_text: str) -> dict:
    # Step 1: deterministic parsing
    parser_result = parse_log(log_text)

    # Step 2: LLM interpretation
    prompt = f"""You are an expert IT infrastructure engineer performing root cause analysis.
    Use ONLY the evidence below. Do not guess. If evidence is insufficient, say so.
    Reply ONLY with valid JSON.

    Parser findings: {json.dumps(parser_result)}
    Raw log excerpt (last 10 lines): {chr(10).join(log_text.strip().split(chr(10))[-10:])}

    Return this exact schema:

    {{
    {
```

```

"probable_root_cause": "...",
"confidence": "HIGH|MEDIUM|LOW",
"affected_component": "...",
"next_action": "specific actionable step",
"prevention": "how to prevent this in future",
"escalate_to_human": true or false
}}"""
response = client.chat.completions.create(
model="gpt-3.5-turbo",
messages=[{"role": "user", "content": prompt}],
temperature=0,
)
rca = json.loads(response.choices[0].message.content)
rca["parser_evidence"] = parser_result
return rca

if __name__ == "__main__":
with open("sample_logs/upgrade_fail.log") as f:
log = f.read()
result = generate_rca(log)
print(json.dumps(result, indent=2))

```

Expected Outputs by End of Day

OUTPUT: Fully working Log Analysis Agent (parser + LLM RCA)

OUTPUT: 3 sample RCA JSON outputs

OUTPUT: Both projects now complete and committed

Interview Questions to Prepare

- ? Walk me through how your Log Analysis Agent processes a log file.
- ? What is the advantage of your hybrid deterministic + LLM design?
- ? How does confidence scoring work in your RCA output?

D21 Polish Both READMEs to Portfolio Standard

GOAL: Write READMEs that make both projects immediately understandable to any engineer who visits your GitHub.

Core Concepts to Understand

- A README is your project's front door — most reviewers spend 60 seconds here before deciding
- Must-have sections: Problem, Architecture, Setup, Usage, Example Output, Limitations, Next Steps
- Architecture diagram in ASCII or text is better than no diagram
- Example output shows the project works — include at least one real JSON sample

Study Resources (read these first)

[How to write a good README — makeareadme.com](https://www.makeareadme.com/) — <https://www.makeareadme.com/>

[GitHub README best practices](https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes) — <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>

Step-by-Step Instructions

1. Write README for automation_support_agent (see template below)
2. Write README for log_analysis_agent
3. Include an architecture diagram (ASCII text is fine)
4. Include one real example JSON output in each README
5. Include setup instructions someone else could follow
6. Commit and push both

Reference Notes

README template sections:

```
## Problem – what IT problem does this solve?
## Architecture – ASCII flow diagram
## Tech Stack – Python, OpenAI SDK, what else?
## Setup – pip install, .env, how to run
## Example Output – paste one real JSON output
## Limitations – what it cannot do yet
## Next Steps – what you would add next
## Author – your background, why you built this
```

Terminal Commands (run in order)

```
code projects/automation_support_agent/README.md
code projects/log_analysis_agent/README.md
git add . && git commit -m 'Day 21: portfolio-ready READMEs for both projects'
git push
```

Expected Outputs by End of Day

OUTPUT: Both READMEs complete and pushed to GitHub

OUTPUT: Both projects are now portfolio-ready

Interview Questions to Prepare

- ? Walk me through your Automation Support Agent architecture.
- ? What are the known limitations of your Log Analysis Agent?

? What would you add if you had two more weeks?

D22 Docker Basics — Containerise Your Thinking

GOAL: Learn the minimum Docker knowledge needed to package and demonstrate your projects professionally.

Core Concepts to Understand

- Image: a snapshot of your app + all its dependencies — build once, run anywhere
- Container: a running instance of an image — isolated from your local machine
- Dockerfile: a recipe that tells Docker how to build the image, step by step
- Why Docker for AI projects: removes 'works on my machine' problems — critical for team environments
- .dockerignore: like .gitignore but for Docker — exclude .env, __pycache__, .venv

Study Resources (read these first)

[Docker Get Started — Official Tutorial](https://docs.docker.com/get-started/) — <https://docs.docker.com/get-started/>

[Docker for Python projects — Real Python](https://realpython.com/docker-in-action-fitter-happier-more-productive/) — <https://realpython.com/docker-in-action-fitter-happier-more-productive/>

Step-by-Step Instructions

1. Verify Docker Desktop is running: `docker --version`
2. Read Docker Get Started Part 1 and Part 2 (30 min)
3. Write the Dockerfile for `automation_support_agent`
4. Write `.dockerignore`
5. Build the image locally (do not run yet — that is Day 23)

Terminal Commands (run in order)

```
docker --version
docker info
touch projects/automation_support_agent/Dockerfile
touch projects/automation_support_agent/.dockerignore
code projects/automation_support_agent/Dockerfile
git add . && git commit -m 'Day 22: Dockerfile for Project 1'
```

Code Samples

Dockerfile

```
# Use official Python slim image
FROM python:3.11-slim

# Set working directory inside container
WORKDIR /app

# Copy requirements first (Docker layer caching)
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Do NOT copy .env — pass secrets as environment variables at runtime
# Environment variables will be passed with: docker run -e OPENAI_API_KEY=...
# Default command
```



```
CMD ["python", "app.py"]
```

.dockerignore

```
.env  
.venv  
__pycache__  
*.pyc  
.git  
*.log
```

Expected Outputs by End of Day

OUTPUT: Dockerfile and .dockerignore for automation_support_agent

OUTPUT: notes/day22-docker-concepts.md with your own Docker definitions

Interview Questions to Prepare

- ? Why does Docker improve reproducibility in AI projects?
 - ? How do you pass secrets like API keys to a Docker container safely?
 - ? What is the difference between a Docker image and a container?
-

D23 Run Project 1 in Docker

GOAL: Build and run your Automation Support Agent inside a Docker container. Verify it works identically to local.

Core Concepts to Understand

- docker build: reads Dockerfile and creates an image
- docker run: starts a container from an image
- -e flag: pass environment variables (like your API key) into the container
- --rm flag: remove the container after it exits (keeps your system clean)
- Layer caching: Docker reuses unchanged layers — put COPY requirements.txt early for faster rebuilds

Study Resources (read these first)

[Docker CLI Reference — run](https://docs.docker.com/reference/cli/docker/container/run/) — <https://docs.docker.com/reference/cli/docker/container/run/>

[Docker Get Started Part 3](https://docs.docker.com/get-started/workshop/03_updating_app/) — https://docs.docker.com/get-started/workshop/03_updating_app/

Step-by-Step Instructions

1. Build the Docker image
2. Run the container — pass your API key as an environment variable
3. Verify the output is identical to running locally
4. If it fails: read the error, fix the Dockerfile, rebuild
5. Document any path or environment adjustments needed

Terminal Commands (run in order)

```
cd projects/automation_support_agent
docker build -t automation-support-agent .
docker run --rm -e OPENAI_API_KEY=$OPENAI_API_KEY automation-support-agent
docker images # verify image is listed
cd ../../
git add . && git commit -m 'Day 23: Project 1 running in Docker'
```

Expected Outputs by End of Day

OUTPUT: Docker image built successfully: automation-support-agent

OUTPUT: Container runs and produces correct output

OUTPUT: notes/day23-docker-run.md with any adjustments made

Interview Questions to Prepare

- ? Walk me through the commands to build and run your agent in Docker.
 - ? How do you ensure secrets are not baked into your Docker image?
 - ? What would you change in the Dockerfile to make it production-ready?
-

D24 Dockerize Project 2 — Log Analysis Agent

GOAL: Apply the same containerisation discipline to the Log Analysis Agent. Build both images and verify both run.

Core Concepts to Understand

- Repeatability: the fact that you can Dockerize two different projects shows a pattern, not a lucky accident
- Shared Dockerfile pattern: notice what is identical between both — extract to a base image if needed
- docker images: lists all images on your machine — keep this list clean
- This gives you two independently deployable, containerised AI agents — strong portfolio signal

Study Resources (read these first)

[Docker Get Started Part 4 — Sharing your image](https://docs.docker.com/get-started/workshop/04_sharing_app/) — https://docs.docker.com/get-started/workshop/04_sharing_app/

Step-by-Step Instructions

1. Write Dockerfile for log_analysis_agent
2. Write .dockerignore for log_analysis_agent
3. Build and run — verify output
4. List both images side by side: docker images
5. Note what was reusable from Project 1's Dockerfile

Terminal Commands (run in order)

```
touch projects/log_analysis_agent/Dockerfile
touch projects/log_analysis_agent/.dockerignore
cd projects/log_analysis_agent
docker build -t log-analysis-agent .
docker run --rm -e OPENAI_API_KEY=$OPENAI_API_KEY log-analysis-agent
docker images
cd ../../
git add . && git commit -m 'Day 24: both projects containerised'
```

Expected Outputs by End of Day

OUTPUT: Both Docker images built and running

OUTPUT: docker images shows both: automation-support-agent, log-analysis-agent

Interview Questions to Prepare

- ? How would you deploy these containers to a cloud environment?
 - ? What is the difference between docker run and docker-compose?
 - ? How would you add health checks to your Docker containers?
-

GOAL: Add a CI pipeline that automatically installs dependencies and runs a smoke test every time you push code.

Core Concepts to Understand

- CI (Continuous Integration): automatically test your code on every push — catch regressions early
- GitHub Actions: GitHub's built-in CI/CD system, free for public repos
- Smoke test: a quick basic test that verifies the project can at least start without crashing
- Green CI badge: visible proof that your code works — shows production mindset to interviewers
- Workflow file: `.github/workflows/python-ci.yml` — YAML file that defines the pipeline steps

Study Resources (read these first)

[GitHub Actions Quickstart](https://docs.github.com/en/actions/quickstart) — <https://docs.github.com/en/actions/quickstart>

[GitHub Actions for Python projects](#) —

<https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python>

Step-by-Step Instructions

1. Create the `.github/workflows/` directory
2. Write `python-ci.yml` (see template below)
3. Write a simple `smoke_test.py` that imports your modules and verifies they load
4. Push and verify the workflow runs green on GitHub
5. Add the CI badge to your root `README.md`

Terminal Commands (run in order)

```
mkdir -p .github/workflows
touch .github/workflows/python-ci.yml
touch smoke_test.py
git add .github/ smoke_test.py
git commit -m 'Day 25: GitHub Actions CI pipeline'
git push
# Check: github.com/YOUR_USERNAME/enterprise-agentic-ai/actions
```

Code Samples

python-ci.yml

```
name: Python CI
on:
  push:
  branches: [ main ]
  pull_request:
  branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
      with:
```

```
python-version: '3.11'
- name: Install dependencies
run: pip install -r requirements.txt
- name: Run smoke test
env:
OPENAI_API_KEY: "test-key-ci-only"
run: python smoke_test.py
```

smoke_test.py

```
"""Smoke test – verifies all modules import without crashing."""
import sys, os
sys.path.insert(0, "projects/automation_support_agent")
sys.path.insert(0, "projects/log_analysis_agent")
# Test 1: tools import
from tools import check_disk_space, check_os_version, TOOL_REGISTRY
result = check_disk_space()
assert "free_gb" in result, "check_disk_space() missing free_gb"
print("PASS: tools.py imports and check_disk_space() works")
# Test 2: parser import
from parser import parse_log
result = parse_log("ERROR disk full\nERROR upgrade aborted")
assert result["has_errors"] == True, "parse_log() should detect errors"
print("PASS: parser.py imports and parse_log() detects errors")
print("\nAll smoke tests passed.")
```

Expected Outputs by End of Day

OUTPUT: Green CI pipeline on GitHub

OUTPUT: CI badge added to README

OUTPUT: smoke_test.py passing

Interview Questions to Prepare

- ? Why do you have a CI pipeline for an AI project?
 - ? What does your smoke test verify?
 - ? How would you add LLM quality evaluation to your CI pipeline?
-

GOAL: Create a HuggingFace Space and prepare one project for live demo deployment.

Core Concepts to Understand

- HuggingFace Spaces: free hosting for AI demos — public URL, no server to manage
- Gradio: Python library that creates a web UI from a function — easiest demo option
- Docker Space: deploys your Docker container directly — more flexible, same HF hosting
- For this project: use Gradio Space — faster setup, looks polished, better for demos
- Live demo = proof you can ship, not just code locally

Study Resources (read these first)

[HuggingFace Spaces Overview](https://huggingface.co/docs/hub/spaces-overview) — <https://huggingface.co/docs/hub/spaces-overview>

[Gradio Quickstart](https://www.gradio.app/guides/quickstart) — <https://www.gradio.app/guides/quickstart>

[Deploy to HF Spaces — Official Guide](https://huggingface.co/docs/hub/spaces-sdks-gradio) — <https://huggingface.co/docs/hub/spaces-sdks-gradio>

Step-by-Step Instructions

1. Create a HuggingFace account at huggingface.co
2. Create a new Space: huggingface.co/new-space — choose Gradio SDK
3. Install Gradio locally
4. Write a simple `gradio_app.py` for the Automation Support Agent
5. Test the Gradio UI locally before deploying

Terminal Commands (run in order)

```
pip install gradio
touch projects/automation_support_agent/gradio_app.py
python projects/automation_support_agent/gradio_app.py
# Open http://127.0.0.1:7860 in your browser
```

Code Samples

`gradio_app.py`

```
import gradio as gr
import sys
sys.path.insert(0, ".")
from app import process_ticket
from report import generate_report
import json

def analyse_ticket(ticket_text: str) -> str:
    result = process_ticket(ticket_text)
    report = generate_report(
        result["ticket"],
        result["classification"],
        result["tool_result"]
    )
    return json.dumps(report, indent=2)

demo = gr.Interface(
    fn=analyse_ticket,
    inputs=gr.Textbox(
        label="Support Ticket",
```

```

placeholder="Describe the IT issue...",
lines=4
),
outputs=gr.Code(label="Analysis Report (JSON)", language="json"),
title="Automation Support Agent",
description="Enterprise IT support analysis powered by Agentic AI.",
examples=[
["My MacBook Pro says the startup disk is almost full"],
["Citrix Workspace crashes immediately on launch"],
["The macOS Sonoma upgrade is stuck at 75%"],
]
)
if __name__ == "__main__":
    demo.launch()

```

Expected Outputs by End of Day

OUTPUT: HuggingFace account and Space created

OUTPUT: Gradio app running locally at <http://127.0.0.1:7860>

OUTPUT: [notes/day26-deployment-plan.md](#)

Interview Questions to Prepare

- ? Why did you choose Gradio for your demo UI?
 - ? What is HuggingFace Spaces and why is it useful for portfolio projects?
 - ? How would deployment differ in a real enterprise environment?
-

D27 Deploy One Project — Go Live

GOAL: Push the Automation Support Agent to HuggingFace Spaces. Get a public URL you can share with anyone.

Core Concepts to Understand

- Deployment checklist: requirements.txt, app.py entrypoint, secrets via HF Space secrets (not in code)
- HF Space secrets: set OPENAI_API_KEY in Space settings — never put it in your code or repo
- app.py must be at the root of the Space repo (or gradio_app.py depending on Space config)
- One live demo is worth more than ten local-only projects when interviewing

Study Resources (read these first)

[HuggingFace Spaces — Adding secrets](https://huggingface.co/docs/hub/spaces-overview#managing-secrets) — <https://huggingface.co/docs/hub/spaces-overview#managing-secrets>

[HuggingFace Spaces — Gradio SDK](https://huggingface.co/docs/hub/spaces-sdks-gradio) — <https://huggingface.co/docs/hub/spaces-sdks-gradio>

Step-by-Step Instructions

1. In your HF Space settings: add OPENAI_API_KEY as a repository secret
2. Create a requirements.txt for the Space (openai, python-dotenv, gradio)
3. Push the code to your HF Space via Git
4. Wait for the Space to build (watch the build logs)
5. Test the live URL with all 3 example tickets
6. Save the live URL in your README and interview notes

Terminal Commands (run in order)

```
# Clone your HF Space repo locally
git clone https://huggingface.co/spaces/YOUR_HF_USERNAME/automation-support-agent hf-space
cd hf-space
# Copy your project files
cp ../projects/automation_support_agent/*.py .
echo 'openai\npython-dotenv\ngradio' > requirements.txt
git add .
git commit -m 'Deploy Automation Support Agent'
git push
cd ..
```

Expected Outputs by End of Day

OUTPUT: Live URL: https://huggingface.co/spaces/YOUR_HF_USERNAME/automation-support-agent

OUTPUT: All 3 example tickets working in the live demo

OUTPUT: Live URL added to your portfolio README

Interview Questions to Prepare

- ? Can you show me a live demo of your project?
- ? How did you handle secrets in your deployed application?
- ? What monitoring would you add to this deployment in production?

D28 Context Engineering Pass

GOAL: Improve prompt quality using context engineering principles. Better context → better outputs without changing the model.

Core Concepts to Understand

- Context engineering: the practice of structuring the information you give an LLM to maximise output quality
- Allowed actions list: explicitly tell the agent what it IS and IS NOT allowed to do in the prompt
- Current evidence block: pass only the relevant, structured evidence — not raw noisy data
- Safe fallback instruction: every prompt must specify what to return when evidence is insufficient
- Context quality often matters more than model choice — a well-structured GPT-3.5 prompt beats a lazy GPT-4 prompt

Study Resources (read these first)

[HuggingFace Context Engineering Course — Unit 1](https://huggingface.co/learn/context-course/unit0/introduction) — <https://huggingface.co/learn/context-course/unit0/introduction>

[HuggingFace Context Engineering Course — Unit 2](https://huggingface.co/learn/context-course/unit0/introduction) — <https://huggingface.co/learn/context-course/unit0/introduction>

Step-by-Step Instructions

1. Read HuggingFace Context Course Unit 1 intro (focus on: what context is, why it matters)
2. Update prompts.py in both projects using the context engineering checklist below
3. Re-test one scenario from each project — note quality improvement
4. Write a context engineering checklist you will use for all future projects

Reference Notes

Context engineering checklist for each prompt:

- | |
|--|
| [] Role is explicit: 'You are an enterprise IT support engineer...' |
| [] Allowed actions listed: 'You may only...' |
| [] Forbidden actions listed: 'You must NEVER...' |
| [] Evidence block is structured JSON, not raw text |
| [] Safe fallback defined: 'If insufficient evidence, return...' |
| [] Output schema is exact: 'Return ONLY valid JSON matching this schema:' |
| [] Example added for complex schemas (few-shot) |

Terminal Commands (run in order)

```
code projects/automation_support_agent/prompts.py
code projects/log_analysis_agent/prompts.py
python projects/automation_support_agent/app.py
python projects/log_analysis_agent/app.py
git add . && git commit -m 'Day 28: context engineering pass on both projects'
```

Expected Outputs by End of Day

OUTPUT: Both prompts.py files updated with context engineering improvements

OUTPUT: notes/day28-context-engineering.md with your personal checklist

Interview Questions to Prepare

- ? What is context engineering and how does it differ from prompt engineering?
- ? How did you improve your agent's output quality on Day 28?
- ? Why is context quality sometimes more important than the model you choose?

GOAL: Convert your 30 days of work into polished interview stories and system design answers.

Core Concepts to Understand

- STAR format for project stories: Situation, Task, Action, Result
- System design for AI: always cover: input handling, model choice, tool layer, guardrails, output, deployment
- The 3 diagrams you must prepare: Automation Support Agent, Log Analysis Agent, and a third future idea
- Interviewers care about: why you made each decision, what went wrong, how you fixed it

Study Resources (read these first)

[OpenAI Agents Guide — full reference](https://openai.github.io/openai-agents-python/) — <https://openai.github.io/openai-agents-python/>

[System design for ML — Made With ML](https://madewithml.com/courses/mlops/systems-design/) — <https://madewithml.com/courses/mlops/systems-design/>

Step-by-Step Instructions

1. Write answers to all 20 interview questions below (see list)
2. Draw 3 architecture diagrams (ASCII or draw.io): Automation Support Agent, Log Analysis Agent, Packaging Assistant idea
3. For each project: prepare a 3-minute spoken walkthrough — record yourself
4. Practise the top 5 questions aloud with someone or record + watch back

Reference Notes

Top 20 Interview Questions to Answer:

1. Tell me about yourself and your career transition.
2. What is an AI Agent and how does it differ from a chatbot?
3. Explain the observe-think-act loop with an IT example.
4. What is RAG and when would you use it?
5. Walk me through your Automation Support Agent architecture.
6. Why did you add an approval gate to your agent?
7. How do you prevent hallucinations in production AI systems?
8. What is the difference between a tool and a prompt?
9. How do you evaluate an AI agent's quality?
10. Walk me through your Log Analysis Agent.
11. Why did you use a hybrid deterministic + LLM design?
12. How does Docker improve your AI project deployment?
13. What does your CI pipeline test?
14. What framework did you choose and why?
15. What are the limitations of your current agents?
16. How would you add RAG to your Automation Support Agent?
17. What monitoring would you add in production?
18. How does your IT background help you build better AI agents?
19. What would you build next if you had 2 more weeks?
20. What is context engineering and why does it matter?

Terminal Commands (run in order)

```
touch notes/top-20-interview-answers.md
code notes/top-20-interview-answers.md
```

```
git add . && git commit -m 'Day 29: interview prep and system design diagrams'
```

Expected Outputs by End of Day

OUTPUT: notes/top-20-interview-answers.md with all 20 answers

OUTPUT: 3 architecture diagrams drawn and saved

OUTPUT: Video recording of 3-minute project walkthroughs

Interview Questions to Prepare

? All 20 questions above — you should now answer any of them confidently.

GOAL: Complete everything. Clean repos. Verify Docker + CI + demo. Update resume. You are now an Agentic AI Engineering candidate.

Core Concepts to Understand

- Portfolio completeness checklist: 2 repos, 1 live demo, 1 CI pipeline, 20 interview answers, 3 diagrams
- Resume bullets must be outcome-oriented: not 'built an agent' but 'built an IT support agent with 85% eval pass rate, deployed on HuggingFace Spaces, with GitHub Actions CI'
- LinkedIn update: add your two projects with links to the live demo and GitHub
- You are not a fresher anymore: you have two shipped AI projects, a CI pipeline, Docker deployment, and 30 days of documented learning

Study Resources (read these first)

[How to write a strong technical resume — levels.fyi](https://www.levels.fyi/blog/engineering-resume-guide.html) — <https://www.levels.fyi/blog/engineering-resume-guide.html>

[GitHub profile README guide](https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/managing-your-profile-readme) — <https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/managing-your-profile-readme>

Step-by-Step Instructions

1. Run the final checklist (see below) — fix anything that is broken
2. Clean both repos: remove debug prints, ensure README is current, verify requirements.txt is complete
3. Verify Docker: build and run both images one final time
4. Verify CI: push a trivial commit and confirm GitHub Actions goes green
5. Verify live demo: open the HF Spaces URL and run all 3 examples
6. Update your resume with 3 new bullet points (see template below)
7. Update your LinkedIn: add both projects with live links
8. Read all your interview notes one final time — you are ready

Reference Notes

Final checklist:

- ☐ Both GitHub repos are clean and public
- ☐ Both READMEs are complete with architecture + example output
- ☐ Docker images build and run for both projects
- ☐ GitHub Actions CI is green
- ☐ HuggingFace Spaces live demo is working
- ☐ 20 interview answers are written and practised
- ☐ 3 architecture diagrams are ready
- ☐ Resume updated with outcome-oriented bullets
- ☐ LinkedIn profile updated

Resume bullets template:

- * Built an Automation Support Agent using OpenAI Agents SDK — classifies IT tickets across 5 categories, invokes diagnostic tools, generates structured JSON reports with human-approval gate. Deployed on HuggingFace Spaces with GitHub Actions CI.
- * Built a Log Analysis Agent using hybrid deterministic regex + LLM design — detects 7 failure patterns, generates root cause analysis with confidence scoring. Containerised with Docker.
- * Completed a structured 30-day Agentic AI engineering program covering LLM APIs, agent frameworks (OpenAI Agents SDK), prompt engineering, context engineering,

Docker, CI/CD, and deployment.

Terminal Commands (run in order)

```
# Final verification
git status # should be clean
docker images # should show both images
python projects/automation_support_agent/app.py
python projects/log_analysis_agent/app.py
git add . && git commit -m 'Day 30: final polish – 30-day roadmap complete'
git push
```

Expected Outputs by End of Day

OUTPUT: Everything on the final checklist is ticked

OUTPUT: Final commit pushed to GitHub

OUTPUT: You are portfolio-ready for Agentic AI Engineering roles

Interview Questions to Prepare

- ? You should now be able to answer every question from Day 29 with confidence.
 - ? Final self-test: record a 5-minute full portfolio walkthrough without notes.
-

Quick Reference: All Verified Study Links

[Python Official Tutorial \(Beginner\)](https://docs.python.org/3/tutorial/index.html) — <https://docs.python.org/3/tutorial/index.html>

[HuggingFace AI Agents Course — Unit 0 Onboarding](https://huggingface.co/learn/agents-course/en/unit0/introduction) — <https://huggingface.co/learn/agents-course/en/unit0/introduction>

[What is an LLM? — IBM explainer](https://www.ibm.com/topics/large-language-models) — <https://www.ibm.com/topics/large-language-models>

[HuggingFace Agents Course — Unit 1: What is an Agent?](https://huggingface.co/learn/agents-course/en/unit1/introduction) — <https://huggingface.co/learn/agents-course/en/unit1/introduction>

[HuggingFace Agents Course — Dummy Agent Library](https://huggingface.co/learn/agents-course/en/unit1/dummy-agent-library) — <https://huggingface.co/learn/agents-course/en/unit1/dummy-agent-library>

[OpenAI: How LLMs work \(gentle intro\)](https://platform.openai.com/docs/introduction) — <https://platform.openai.com/docs/introduction>

[OpenAI Prompt Engineering Guide](https://platform.openai.com/docs/guides/prompt-engineering) — <https://platform.openai.com/docs/guides/prompt-engineering>

[OpenAI Help: Best Practices for Prompt Engineering](https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api) — <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>

[OpenAI Python SDK — Getting Started](https://platform.openai.com/docs/libraries) — <https://platform.openai.com/docs/libraries>

[OpenAI API Reference — Chat Completions](https://platform.openai.com/docs/api-reference/chat) — <https://platform.openai.com/docs/api-reference/chat>

[python-dotenv docs](https://pypi.org/project/python-dotenv/) — <https://pypi.org/project/python-dotenv/>

[HuggingFace Agents Course — Unit 1: Tools](https://huggingface.co/learn/agents-course/en/unit1/tools) — <https://huggingface.co/learn/agents-course/en/unit1/tools>

[HuggingFace Agents Course — Agent Loop explained](https://huggingface.co/learn/agents-course/en/unit1/agent-steps-and-structure) — <https://huggingface.co/learn/agents-course/en/unit1/agent-steps-and-structure>

[OpenAI Agents Guide](https://openai.github.io/openai-agents-python/) — <https://openai.github.io/openai-agents-python/>

[OpenAI Chat Completions API](https://platform.openai.com/docs/guides/text-generation) — <https://platform.openai.com/docs/guides/text-generation>

[Git and GitHub for Beginners — freeCodeCamp](https://www.freecodecamp.org/news/git-and-github-for-beginners/) — <https://www.freecodecamp.org/news/git-and-github-for-beginners/>

[GitHub Quickstart](https://docs.github.com/en/get-started/quickstart/hello-world) — <https://docs.github.com/en/get-started/quickstart/hello-world>

[OpenAI Agents SDK — Quickstart](https://openai.github.io/openai-agents-python/quickstart/) — <https://openai.github.io/openai-agents-python/quickstart/>

[smolagents Documentation \(for comparison\)](https://huggingface.co/docs/smolagents/index) — <https://huggingface.co/docs/smolagents/index>

[OpenAI Agents Guide — Guardrails](https://openai.github.io/openai-agents-python/guardrails/) — <https://openai.github.io/openai-agents-python/guardrails/>

[OpenAI Agents Guide — Running Agents](https://openai.github.io/openai-agents-python/running_agents/) — https://openai.github.io/openai-agents-python/running_agents/

[OpenAI Structured Outputs](https://platform.openai.com/docs/guides/structured-outputs) — <https://platform.openai.com/docs/guides/structured-outputs>

[OpenAI Agents SDK — Tools](https://openai.github.io/openai-agents-python/tools/) — <https://openai.github.io/openai-agents-python/tools/>

[Python type hints and docstrings — Real Python](https://realpython.com/python-type-checking/) — <https://realpython.com/python-type-checking/>

[Writing good test cases — Google Testing Blog](https://testing.googleblog.com/) — <https://testing.googleblog.com/>

[OpenAI: Safety in building agents](https://developers.openai.com/api/docs/guides/agents) — <https://developers.openai.com/api/docs/guides/agents>

[HuggingFace Context Course — Unit 1](https://huggingface.co/learn/context-course/unit0/introduction) — <https://huggingface.co/learn/context-course/unit0/introduction>

[How to evaluate LLM outputs — Weights & Biases guide](https://wandb.ai/site/articles/evaluate-llm) — <https://wandb.ai/site/articles/evaluate-llm>

[Python re module — Regular Expressions](https://docs.python.org/3/library/re.html) — <https://docs.python.org/3/library/re.html>

[Regex tutorial for Python — Real Python](https://realpython.com/regex-python/) — <https://realpython.com/regex-python/>

[How to write a good README — makeareadme.com](https://www.makeareadme.com/) — <https://www.makeareadme.com/>

[GitHub README best practices](https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes) — <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>

[Docker Get Started — Official Tutorial](https://docs.docker.com/get-started/) — <https://docs.docker.com/get-started/>

[Docker for Python projects — Real Python](https://realpython.com/docker-in-action-fitter-happier-more-productive/) — <https://realpython.com/docker-in-action-fitter-happier-more-productive/>

[Docker CLI Reference — run](https://docs.docker.com/reference/cli/docker/container/run/) — <https://docs.docker.com/reference/cli/docker/container/run/>

[Docker Get Started Part 3](https://docs.docker.com/get-started/workshop/03_updating_app/) — https://docs.docker.com/get-started/workshop/03_updating_app/

[Docker Get Started Part 4 — Sharing your image](https://docs.docker.com/get-started/workshop/04_sharing_app/) — https://docs.docker.com/get-started/workshop/04_sharing_app/

[GitHub Actions Quickstart](https://docs.github.com/en/actions/quickstart) — <https://docs.github.com/en/actions/quickstart>

[GitHub Actions for Python projects](https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python) — <https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-python>

[HuggingFace Spaces Overview](https://huggingface.co/docs/hub/spaces-overview) — <https://huggingface.co/docs/hub/spaces-overview>

[Gradio Quickstart](https://www.gradio.app/guides/quickstart) — <https://www.gradio.app/guides/quickstart>

[Deploy to HF Spaces — Official Guide](https://huggingface.co/docs/hub/spaces-sdks-gradio) — <https://huggingface.co/docs/hub/spaces-sdks-gradio>

[HuggingFace Spaces — Adding secrets](https://huggingface.co/docs/hub/spaces-overview#managing-secrets) — <https://huggingface.co/docs/hub/spaces-overview#managing-secrets>
[System design for ML — Made With ML](https://madewithml.com/courses/mlops/systems-design/) — <https://madewithml.com/courses/mlops/systems-design/>
[How to write a strong technical resume — levels.fyi](https://www.levels.fyi/blog/engineering-resume-guide.html) — <https://www.levels.fyi/blog/engineering-resume-guide.html>
[GitHub profile README guide](https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/managing-your-profile-readme) — <https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/managing-your-profile-readme>

All Terminal Commands — Day by Day

Day 01: Environment Setup & Python Foundations

```
mkdir enterprise-agentic-ai
cd enterprise-agentic-ai
python3 -m venv .venv
source .venv/bin/activate # Mac/Linux
# .venv\Scripts\activate # Windows
pip install --upgrade pip
mkdir -p notes/projects/playground
touch notes/learning-journal.md
touch notes/interview-notes.md
touch notes/day01-foundations.md
code notes/day01-foundations.md
```

Day 02: LLM & Agent Concepts Deep Dive

```
touch notes/day02-agent-concepts.md
code notes/day02-agent-concepts.md
```

Day 03: Prompt Engineering for IT Support

```
touch notes/day03-prompts.md
code notes/day03-prompts.md
```

Day 04: First LLM API Calls in Python

```
pip install openai python-dotenv
touch .env .gitignore
echo 'OPENAI_API_KEY=your_key_here' >> .env
echo '.env' >> .gitignore
mkdir -p projects/playground
touch projects/playground/simple_completion.py
touch projects/playground/classify_issue.py
touch projects/playground/summarize_log.py
python projects/playground/simple_completion.py
python projects/playground/classify_issue.py
```

Day 05: The Agent Loop — Understand & Map

```
touch notes/day05-agent-loop.md
code notes/day05-agent-loop.md
```

Day 06: Interview Framing + Build Your First Chatbot

```
mkdir -p projects/chatbot
touch projects/chatbot/chatbot.py
python projects/chatbot/chatbot.py
```

Day 07: Week 1 Review & First GitHub Push

```
cd enterprise-agentic-ai
```

```
git init
touch README.md
git add .
git commit -m 'Week 1: foundations, chatbot, prompt examples'
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/enterprise-agentic-ai.git
git push -u origin main
```

Day 08: Choose Framework & Create Project Skeleton

```
pip install openai-agents
pip freeze > requirements.txt
mkdir -p projects/automation_support_agent/sample_data
touch projects/automation_support_agent/app.py
touch projects/automation_support_agent/tools.py
touch projects/automation_support_agent/prompts.py
touch projects/automation_support_agent/report.py
touch projects/automation_support_agent/requirements.md
touch projects/automation_support_agent/README.md
touch projects/automation_support_agent/eval_cases.md
git add . && git commit -m 'Day 8: project skeleton for Automation Support Agent'
```

Day 09: Define Enterprise Requirements

```
code projects/automation_support_agent/requirements.md
```

Day 10: Build the Issue Classifier

```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 10: issue classifier working'
```

Day 11: Build Diagnostic Tools

```
code projects/automation_support_agent/tools.py
python -c "from projects.automation_support_agent.tools import check_disk_space;
print(check_disk_space())"
git add . && git commit -m 'Day 11: diagnostic tools layer'
```

Day 12: Connect Classifier to Tools — First End-to-End Run

```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 12: end-to-end classifier-to-tool flow'
```

Day 13: Build Structured Support Summaries

```
code projects/automation_support_agent/report.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 13: structured report generation'
```

Day 14: Create Realistic Sample Cases

```
code projects/automation_support_agent/sample_data/disk_issue.txt
code projects/automation_support_agent/sample_data/upgrade_failure.log
code projects/automation_support_agent/sample_data/package_install.log
code projects/automation_support_agent/sample_data/citrix_issue.log
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 14: realistic sample cases added'
```

Day 15: Add Approval Gate (Human-in-the-Loop)


```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 15: approval gate added – human-in-the-loop'
```

Day 16: Strengthen Prompts for Enterprise Use

```
code projects/automation_support_agent/prompts.py
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 16: enterprise-safe prompts with anti-hallucination constraints'
```

Day 17: Add Evaluation Cases — Test Like an Engineer

```
code projects/automation_support_agent/eval_cases.md
python projects/automation_support_agent/app.py
git add . && git commit -m 'Day 17: 15 eval cases with pass/fail results'
```

Day 18: Start Project 2 — Log Analysis Agent

```
mkdir -p projects/log_analysis_agent/sample_logs
touch projects/log_analysis_agent/app.py
touch projects/log_analysis_agent/parser.py
touch projects/log_analysis_agent/prompts.py
touch projects/log_analysis_agent/README.md
touch projects/log_analysis_agent/sample_logs/upgrade_fail.log
touch projects/log_analysis_agent/sample_logs/disk_full.log
touch projects/log_analysis_agent/sample_logs/signing_error.log
touch projects/log_analysis_agent/sample_logs/permission_denied.log
touch projects/log_analysis_agent/sample_logs/timeout.log
git add . && git commit -m 'Day 18: Log Analysis Agent skeleton'
```

Day 19: Build the Log Parser

```
code projects/log_analysis_agent/parser.py
python projects/log_analysis_agent/parser.py
git add . && git commit -m 'Day 19: log parser with pattern detection'
```

Day 20: Add LLM-Based RCA Summary

```
code projects/log_analysis_agent/app.py
python projects/log_analysis_agent/app.py
git add . && git commit -m 'Day 20: LLM RCA complete – Log Analysis Agent working'
```

Day 21: Polish Both READMEs to Portfolio Standard

```
code projects/automation_support_agent/README.md
code projects/log_analysis_agent/README.md
git add . && git commit -m 'Day 21: portfolio-ready READMEs for both projects'
git push
```

Day 22: Docker Basics — Containerise Your Thinking

```
docker --version
docker info
touch projects/automation_support_agent/Dockerfile
touch projects/automation_support_agent/.dockerignore
code projects/automation_support_agent/Dockerfile
git add . && git commit -m 'Day 22: Dockerfile for Project 1'
```

Day 23: Run Project 1 in Docker

```
cd projects/automation_support_agent
```

```

docker build -t automation-support-agent .
docker run --rm -e OPENAI_API_KEY=$OPENAI_API_KEY automation-support-agent
docker images # verify image is listed
cd ../../
git add . && git commit -m 'Day 23: Project 1 running in Docker'

```

Day 24: Dockerize Project 2 — Log Analysis Agent

```

touch projects/log_analysis_agent/Dockerfile
touch projects/log_analysis_agent/.dockerignore
cd projects/log_analysis_agent
docker build -t log-analysis-agent .
docker run --rm -e OPENAI_API_KEY=$OPENAI_API_KEY log-analysis-agent
docker images
cd ../../
git add . && git commit -m 'Day 24: both projects containerised'

```

Day 25: GitHub Actions CI Pipeline

```

mkdir -p .github/workflows
touch .github/workflows/python-ci.yml
touch smoke_test.py
git add .github/ smoke_test.py
git commit -m 'Day 25: GitHub Actions CI pipeline'
git push
# Check: github.com/YOUR_USERNAME/enterprise-agentic-ai/actions

```

Day 26: Hugging Face Spaces — Choose Deployment Target

```

pip install gradio
touch projects/automation_support_agent/gradio_app.py
python projects/automation_support_agent/gradio_app.py
# Open http://127.0.0.1:7860 in your browser

```

Day 27: Deploy One Project — Go Live

```

# Clone your HF Space repo locally
git clone https://huggingface.co/spaces/YOUR_HF_USERNAME/automation-support-agent hf-space
cd hf-space
# Copy your project files
cp ../projects/automation_support_agent/*.py .
echo 'openai\npython-dotenv\ngradio' > requirements.txt
git add .
git commit -m 'Deploy Automation Support Agent'
git push
cd ..

```

Day 28: Context Engineering Pass

```

code projects/automation_support_agent/prompts.py
code projects/log_analysis_agent/prompts.py
python projects/automation_support_agent/app.py
python projects/log_analysis_agent/app.py
git add . && git commit -m 'Day 28: context engineering pass on both projects'

```

Day 29: Interview & System Design Preparation

```

touch notes/top-20-interview-answers.md
code notes/top-20-interview-answers.md
git add . && git commit -m 'Day 29: interview prep and system design diagrams'

```

Day 30: Final Polish & Portfolio Pack

```
# Final verification
git status # should be clean
docker images # should show both images
python projects/automation_support_agent/app.py
python projects/log_analysis_agent/app.py
git add . && git commit -m 'Day 30: final polish – 30-day roadmap complete'
git push
```